

Ce qu'on a construit

1 / 7 — PREMIÈRE IMPLÉMENTATION

Le socle du registre : une autorité de rôles, deux stockages permanents, un module métier



CE QUI TOURNE

- **5 contrats Solidity** — 0.8.34, version figée, compilation propre
- **Le passeport ERC-721** non transférable, avec ses traits figés à la fabrication
- **Les lots de matériaux ERC-1155**, brûlés dans la transaction même du mint
- **Le cycle de vie complet** — fabrication, conformité, pose
- **4 tests de bout en bout** au vert, linter à zéro problème



CE QUI N'EST PAS FAIT

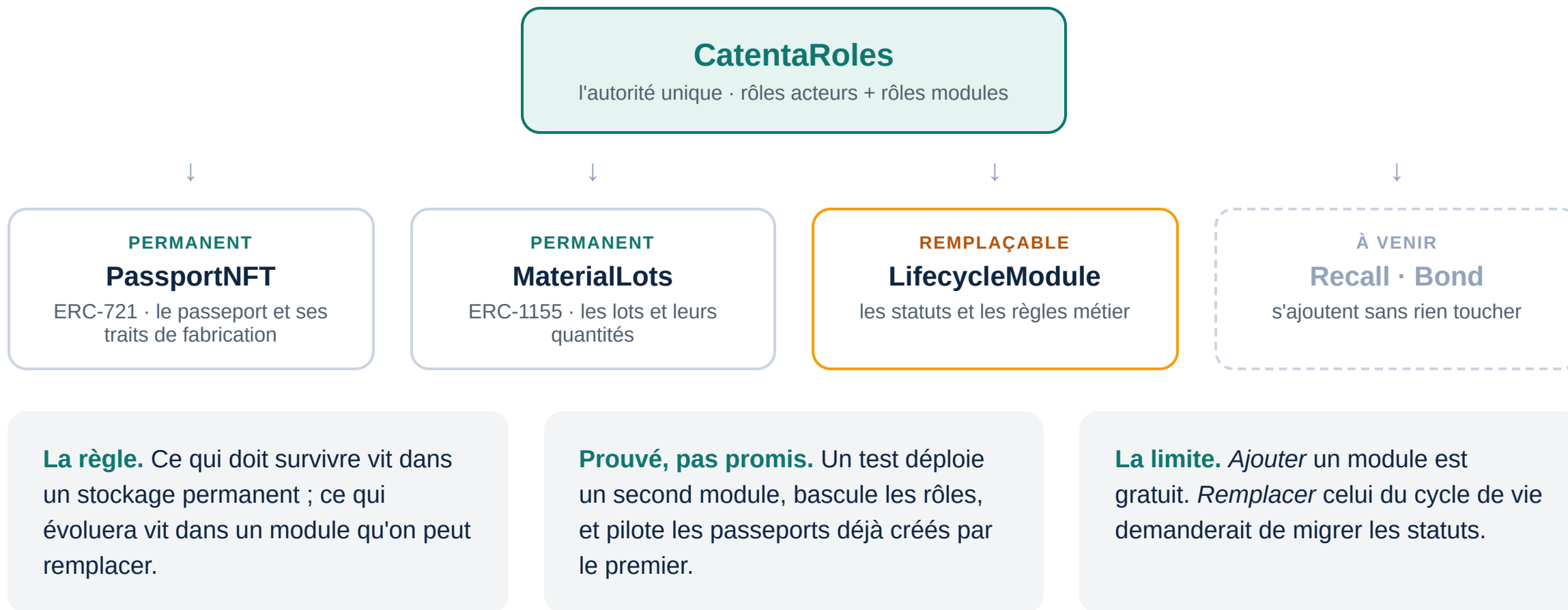
- **Le rappel de lot** et les accusés de réception
- **La caution qualité ERC-20** et son slashing
- **L'intégration continue** — aucune CI à ce jour
- **Le déploiement** sur réseau de test et sa vérification
- **Le front** de consultation multi-rôles

Socle vérifiable · 5 contrats · 2 standards ERC · 4 tests au vert

L'architecture en trois couches

2 / 7 — PREMIÈRE IMPLÉMENTATION

Personne ne connaît l'adresse de personne : tout passe par les rôles



Autorité · Stockage permanent · Module remplaçable · Découplage par les rôles

Les rôles — qui peut quoi

3 / 7 — PREMIÈRE IMPLÉMENTATION

Deux familles volontairement séparées : les humains d'un côté, les contrats de l'autre



RÔLES ACTEURS

- **LAB_ROLE** — laboratoire : déclare les lots, mint les passeports, déposera la caution
- **PRACTITIONER_ROLE** — praticien : atteste la conformité, enregistre la pose
- **DISTRIBUTOR_ROLE** — dépôt dentaire : accusera réception des rappels
- **REGULATOR_ROLE** — Ordre et ARS : lecture totale, déclarera les rappels
- **DEFAULT_ADMIN_ROLE** — le consortium : attribue et révoque les rôles



RÔLES TECHNIQUES

- **PASSPORT_MINTER · PASSPORT_CONTROLLER** — créer un passeport, autoriser un transfert
- **LOT_MINTER · LOT_BURNER** — déclarer un lot, consommer de la matière
- **Accordés à des contrats, jamais à des personnes.** C'est ce qui rend un module ajoutable en une transaction
- **Le point de vigilance :** accorder un rôle technique à une adresse humaine court-circuiterait toute la logique métier — la parade est le multisig, pas le code

AccessControl OpenZeppelin · agrément sur liste blanche · un seul point d'autorité

Les types de token

4 / 7 — PREMIÈRE IMPLÉMENTATION

Deux standards implémentés, un troisième à venir — aucun décoratif



ERC-721 — LE PASSEPORT

- **Un token = un dispositif.** Couronne, bridge, implant
- **Non transférable (soulbound)** — un dispositif médical est lié à un patient, ce n'est pas un actif échangeable
- **Traits figés au mint** : lot d'origine, horodatage, empreinte du dossier de conformité — aucun setter, et indestructible



ERC-1155 — LES LOTS

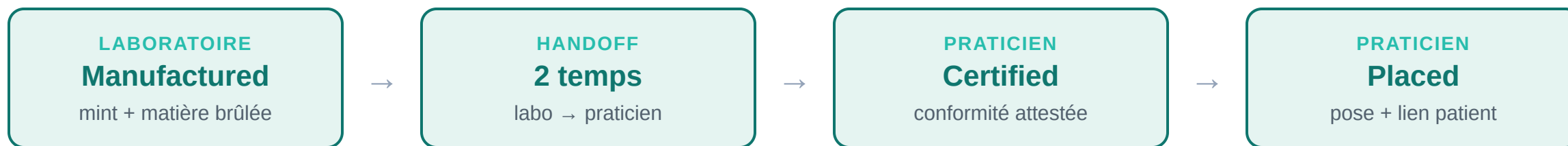
- **Un identifiant par lot, une quantité par lot** — « X grammes de zircone du lot N »
- **Brûlé à la fabrication**, dans la transaction même qui crée le passeport : le lien matière → prothèse ne peut pas être reconstruit après coup
- **Quantité restante native**, et lot non transférable : il appartient au laboratoire qui l'a déclaré

À venir — ERC-20, la caution qualité. Le laboratoire immobilise une valeur, saisissable par le régulateur en cas de rappel. Le registre acceptera **un jeton qu'il n'émet pas** : une garantie libellée dans sa propre monnaie ne garantit rien.

Le cycle de vie d'un passeport

5 / 7 — PREMIÈRE IMPLÉMENTATION

Une machine à états à sens unique : aucune étape ne peut être sautée, rejouée ou inversée



Chaque acte est attribué. Qui, quand, et sous quel rôle agréé — horodaté par la chaîne, non réécrivable ensuite.

Détenir le token vaut preuve de garde. Un praticien ne peut attester que ce qu'il a physiquement reçu.

Le patient n'a pas de wallet. Le passeport reste au cabinet ; le lien patient est un engagement **salé**, dont le sel vit hors chaîne et peut être détruit.

Machine à états déterministe · gardes de rôle et de statut sur chaque écriture

Deux choix qui méritent l'explication ^{0 / 7 — PREMIÈRE IMPLÉMENTATION}

Les deux endroits où la conception d'origine avait un défaut



LE HANDOFF EN DEUX TEMPS

- **Le piège** : un simple drapeau « transfert autorisé » que personne ne remet à zéro laisse le passeport transférable **indéfiniment** — le verrou se désactive en silence
- **La correction** : le détenteur désigne un destinataire nominatif, qui doit accepter. L'autorisation est consommée par le transfert lui-même
- **Le coût assumé** : deux transactions au lieu d'une



LE RAPPEL SERA DÉRIVÉ

- **Le piège** : notifier N passeports lors d'un rappel, c'est une boucle qui explose en coût — et devient impossible à exécuter
- **La correction** : on marque **le lot**, une seule écriture. Le statut « rappelé » de chaque passeport est calculé à la lecture
- **Le coût assumé** : la chaîne ne notifie personne. Elle rend l'information disponible instantanément et prouve qui l'a lue

Verrou non contournable · coût de rappel constant · aucune boucle on-chain

Où on en est

7 / 7 — PREMIÈRE IMPLÉMENTATION

Le socle est solide — il n'est pas encore démontrable de bout en bout

ACQUIS

- L'architecture modulaire, **vérifiée par un test** et non seulement décrite
- Les deux stockages permanents, construits pour ne jamais être redéployés
- Le verrou du passeport, posé là où aucun chemin de transfert ne l'évite
- Le lien matière → prothèse, atomique

À FAIRE, PAR ORDRE DE VALEUR

- **L'intégration continue** — plus simple à monter sur 5 contrats que sur 9
- **La vraie suite de tests** — aujourd'hui les chemins d'erreur ne sont pas couverts
- **Le module de rappel** — il ne touchera aucun contrat existant
- **Déploiement puis front** — sans eux, rien n'est démontrable

Un chiffre à ne pas surinterpréter. La couverture de tests affiche 95 % de lignes — avec 4 tests seulement. Elle mesure les lignes exécutées, pas les comportements *vérifiés* : le chemin nominal est verrouillé, les chemins d'erreur ne le sont pas encore.

Socle vérifiable · limites énoncées · prochaine étape : CI et couverture réelle